

PRE-CLASS READING 5 OF 5

Claim, evidence, correction — Devin and the two years after the demo

Cognition Labs (2024). “Introducing Devin, the first AI software engineer.” Technical blog, 12 March 2024 — read together with the independent evaluation that followed (Answer.AI, 8 January 2025) and Cognition’s own published course-correction (2025–26). The most instructive arc in the agent field: a spectacular claim, a hard external test, and what the company learned in public.

ASSIGNED READING

Read the original “Introducing Devin” post (≈ 8 min — it is short, and watch at least one demo video). Then read Answer.AI’s “Thoughts On A Month With Devin” (≈ 15 min). If you have time, skim Cognition’s “Don’t Build Multi-Agents” (June 2025) and its 2026 successor. Read the three as a sequence, not as three opinions.

» source ~25 min • this summary + reasoning notes ~20 min

Note on currency. The March 2024 post is a historical document — do not judge Devin-in-2026 by it. Cognition has shipped continuously since: **Devin 2.0** (April 2025, with an agent-native IDE and a US\$20 entry plan), **Devin 2.1** (May 2025, task-confidence indicators), the **Windsurf acquisition** (July 2025), their own **SWE-1.5/1.6/1.7** coding models (Oct 2025 – Jul 2026), **Devin 2.2** (February 2026, described by the company as its most significant update since launch), and the **FrontierCode** benchmark (June 2026). The point of this reading is not to score Devin. It is to watch a claim meet an evaluation, and to see what the company did next.

KEY TAKEAWAY

In March 2024 Cognition announced “the first AI software engineer” and backed it with a headline number: Devin resolved **13.86%** of SWE-bench issues end-to-end **unassisted**, against a previous state of the art of **1.96%** — and the comparison was not like-for-like in the usual direction, because “Devin was unassisted, whereas all other models were assisted (meaning the model was told exactly which files need to be edited).” It was a genuine result and a seven-fold jump. Ten months later Answer.AI ran twenty real tasks and reported **3 successes, 14 failures and 3 inconclusive**, with a devastating summary: the tasks it could do were “so small and well-defined that I may as well do them myself, faster, my way.” Both are true. **A benchmark measures capability on a curated distribution; a field evaluation measures usefulness on yours** — and the gap between them is where most enterprise AI budgets are lost. What makes this an honest teaching case rather than a hit piece is what followed: Cognition published its own reversals, narrowed its claims, changed the product shape, and by 2026 was writing about which multi-agent patterns actually work rather than which ones sound impressive.

WHY IT MATTERS FOR THIS COURSE

This is the reading that puts a price on everything else in the set. Reading 1 gave you restraint as doctrine (defined in its appendix); here is what happens without it. Reading 2 gave you the loop; here are its failure modes at full scale, in a real product, judged by strangers. Reading 4 gave you an agent as a priced product; this is the same bet made two years earlier and marked to market. For Capsule 2 it supplies the failure taxonomy you will deliberately reproduce in the notebook. For Capsule 3 and for your **A2**, it supplies the single most useful professional habit in this course: when someone shows you an agent demo, ask **what it was evaluated on, by whom, and on whose tasks** — and know that the answer “SWE-bench” and the answer “our twenty tasks” are different answers to different questions.

Feeds Class 4: C1 (autonomy as a slider, not a switch) • C2 (the real failure taxonomy • why evals must use your tasks) • C3 (cost of failed autonomous runs • the case discussion).

MAIN POINTS

The 2024 claims, the Answer.AI results and Cognition’s own posts are the sources. Anything after Our reading is this brief’s interpretation — the “claim, evidence, correction” arc, the autonomy slider and the five vendor questions are ours.

The claim (12 March 2024)

- **The number.** 13.86% of SWE-bench issues resolved end-to-end unassisted, vs 1.96% prior SOTA — a real result, correctly caveated in the post itself about the assisted/unassisted asymmetry.
- **The framing.** “A tireless, skilled teammate, equally ready to build alongside you or independently complete tasks for you to review.” Note the two modes bundled into one sentence: collaborator and autonomous worker. Almost all the subsequent disappointment lives in the second.
- **The capability list.** Learn unfamiliar technologies, build and deploy apps end-to-end, find and fix bugs autonomously, train and fine-tune models, contribute to mature production repositories, and complete real paid jobs on Upwork. Delivered with shell, code editor and browser inside a sandboxed environment.

- **Why it landed so hard.** It was the first credible demonstration that a loop plus tools plus a sandbox could carry a task for thousands of decisions rather than a handful. Read charitably, the demo was true and the extrapolation was the problem — and the extrapolation was done as much by the audience as by the company.

The evidence (Answer.AI, 8 January 2025)

- **The method.** Twenty real tasks over about a month, by an experienced engineering team, on their own work — not a benchmark suite. Results: **3 successes, 14 failures, 3 inconclusive.**
- **The shape of the failures is the finding.** New standalone projects: 2 successes out of 8. Research tasks: 1 of 4. **Analysing existing code: 0 of 4. Modifying existing projects: 0 of 4.** In other words, it worked on greenfield and failed on brownfield — and essentially all enterprise software work is brownfield.
- **Named failure modes.** *Overcomplexity* — “spaghetti code” with unnecessary abstraction layers. *Hallucinated capabilities* — it invented platform features that do not exist and then spent days pursuing them. *Context blindness* — it could not absorb project-specific tooling even with the documentation available. *False positives* — a security review that flagged non-existent vulnerabilities. *Tunnel vision* — an inability to notice that the problem lay outside the area it was working in.
- **Map each one back to a mechanism you now understand.** *Hallucinated capabilities* = no ground truth for that claim, so nothing ever contradicted it (Reading 1). *Days spent pursuing an impossible fix* = no world model, so “what happens if I try this?” could only be answered by trying (Reading 3). *Context blindness* = the documentation existed but never entered the context; an observation failure, not a reasoning one (Reading 2). *False positives* = reasoning that overrides evidence, the prior winning over what the tool actually returned (Reading 2). *Spaghetti and tunnel vision* = compounding drift, locally reasonable steps adding up to something globally wrong, with no gate to catch it (Reading 1). This is the whole week, in a field report.
- **The conclusion that stings.** No reliable niche was found: the tasks it could do were small enough to do yourself. Their preference was for developer-driven tools allowing incremental correction — which is precisely the autonomy-slider argument, arrived at empirically.

The correction (2025–2026)

- **Product shape changed.** Devin 2.0 (Apr 2025) moved to an **agent-native IDE** — a human-in-the-loop surface rather than a fire-and-forget worker — at a lower entry price. Devin 2.1 (May 2025) added **confidence indicators** on task completion, which is an admission with a UI: the system now tells you how much to trust each result rather than asserting success.
- **“Don’t Build Multi-Agents” (Walden Yan, 12 June 2025).** Two principles: **share context** — “share full agent traces, not just individual messages” — and **actions carry implicit decisions, and conflicting decisions carry bad results.** Parallel subagents each make unstated assumptions; nothing reconciles them, and the main agent cannot integrate the mismatched pieces. Their worked example is a Flappy Bird clone where one subagent builds a Mario-style background and another builds a mismatched bird.
- **“Multi-Agents: What’s Actually Working” (Walden Yan, 22 April 2026).** The reversal, and note its precise shape — not “we were wrong”, but “a narrower class works”: **multiple agents may contribute intelligence, while writes stay single-threaded.** What works: a **review agent** with clean context reading another agent’s pull request (reported at roughly 2 bugs per PR, 58% of them severe); a **“smart friend”** pattern where a cheaper primary model escalates hard sub-problems to a frontier model; and a **manager Devin** that decomposes a project and coordinates child agents through an internal MCP. Context given for the change: models became “way more naturally agentic”, and enterprise adoption of Devin grew roughly **8× over six months.**
- **They also built their own measuring stick.** After the SWE-bench technical report (2024), Cognition shipped **FrontierCode** (June 2026), a benchmark asking whether models “can actually write good code” rather than merely resolve issues — a direct response to the 2024-era lesson that the benchmark you win is not necessarily the thing your customers wanted.

The transferable lesson: autonomy is a slider

- **The 2024 framing was binary.** Two options were on offer, and nothing between them: supervise a tool keystroke by keystroke (autocomplete, Copilot), or hand the whole task over and inspect only the result. Devin’s pitch was that AI had crossed from the first to the second — not a better tool, a teammate you assign work to. Its own launch sentence offered both poles at once and no dial. Everything since has been the discovery of that middle: propose-then-approve, plan review before execution, confidence signals, review agents, single-threaded writes with parallel advice. The best 2026 agent products let you **choose where on the slider each task sits**, and that is a product decision as much as a technical one.
- **Where the value actually accrued.** Not in more autonomy, but in better harnesses, better context management, better tools, and better evals — exactly Reading 1’s 2025–26 posts. The frontier moved; the doctrine of restraint held.

READING FOCUS QUESTIONS — bring your answers to class

There are no wrong answers here — these surface how you think, not what you recall. A clue sits under each question to give you a direction to push on; the fuller reasoning is in the section after.

- 13.86% on SWE-bench and 3-out-of-20 in the field are both honest numbers about the same system. Explain how, without accusing anyone of dishonesty.

Clue. Compare what each measurement actually sampled, not how impressive each sounds. Ask three things of both: who chose the tasks, what counts as success, and how similar those tasks are to the ones you would bring. Notice that the benchmark tasks came with a specification and a test that decides the outcome — and then ask how many of your own work tasks arrive that way.

- Answer.AI found 0 successes out of 8 on “analysing existing code” and “modifying existing projects”, but 2 of 8 on new projects. What property of brownfield work breaks the agent — and does that property have a fix in 2026?

Clue. Start from what a new project does not have: no existing conventions, no hidden coupling, nothing already depending on your choices being right. Then list what an agent would need in order to acquire all of that, and notice you have just described the context problem — the constraints are real but mostly unwritten. Ask whether the 2026 answers (bigger context windows, codemaps, retrieval, harnesses, memory files) address the unwritten part or only the unread part.

- Cognition said “don’t build multi-agents” in June 2025 and “here is the multi-agent pattern that works” in April 2026. Is that a contradiction, a correction, or a refinement — and what should you conclude about advice with a 2025 date on it?

Clue. Read the two claims precisely enough to see whether they can both be true: the 2026 rule permits multiple agents to contribute *intelligence* while forbidding parallel *writes*, and that is exactly the failure the 2025 post diagnosed. Then ask what changed underneath — capability, or the argument. The meta-lesson is about which parts of technical advice have a shelf life and which do not.

- The 2026 review-agent pattern is: a second agent with clean context reads the first agent’s output. Why does a clean context find bugs the original agent cannot — and where else would you apply this?

Clue. Think about what the original agent is carrying that the reviewer is not: every justification it wrote for its own choices, sitting in its context and conditioning what it says next. Connect that to Reading 1’s context rot, and to why human code review works even when the reviewer is no better than the author. Then look for other places where the reviewer’s ignorance is the asset.

- You are about to be shown an agent demo by a vendor. Write the five questions you will ask — and say which single answer would most change your decision.

Clue. Build the questions from this week’s failure modes rather than from a generic procurement list: benchmark vs your tasks, brownfield vs greenfield, what happens on failure, who pays for a failed run, and how the system tells you it is unsure. Then pick the one whose answer you cannot verify any other way — for most buyers that is not a capability question at all, it is about what the system does when it is wrong.

HOW TO THINK ABOUT THESE — read after you have had a go

These are not model answers to be memorised — they are the line of reasoning we will pick up in class. Class time is short, so they are written to stand on their own.

Q1 — Two honest numbers, two different questions

Our reading. A benchmark answers: *on this fixed, curated set of tasks, with success defined by a test, how often does the system succeed?* SWE-bench issues come pre-specified, with a reproducible environment and a test that decides the outcome — the ground-truth signal Reading 1 says an agent needs is supplied by construction. A field evaluation answers a different question: *on the work we happen to have, described the way we happen to describe it, does this save us time?* Those tasks arrive underspecified, embedded in a codebase full of unwritten constraints, and success is judged by an engineer who has an opinion about maintainability. Neither measurement is dishonest; they sample different distributions and use different success functions. The professional habit that follows is simple and worth a great deal: **a benchmark result is a lower bound on capability and tells you almost nothing about usefulness.** Ask who chose the tasks, and how they resemble yours. It is also why Anthropic’s eval advice (Reading 1) says to build your set from **real failures you observed**, and why Cognition eventually built its own benchmark — winning someone else’s benchmark is not the same as being good at the job.

Q2 — Brownfield is mostly unwritten constraints

Our reading. A new project has no history, so almost any coherent choice is acceptable — which is precisely why greenfield tasks are where an agent looks best, and why demos are always greenfield. Existing systems are the opposite: they are a dense web of constraints, most of them undocumented. This module is only called from that scheduler. That table has a consumer nobody remembers. This pattern was chosen because of an incident in 2023. The agent cannot read any of that, so it produces changes that are locally reasonable and globally wrong — which is exactly the “spaghetti” and “tunnel vision” Answer.AI described. Have the 2026 tools fixed it? Partly. Larger context

windows, retrieval over the repository, codemaps, harness files and persistent notes all attack the *unread* part of the problem — constraints that exist in writing somewhere. They do not touch the *unwritten* part, and the unwritten part is the majority. Which yields a genuinely useful and slightly uncomfortable corollary for your organisation: **the return on an agent is bounded by how much of your institutional knowledge is written down**. Teams with real documentation, tests and conventions get more from agents — not because the agent is smarter there, but because more of the constraint set is legible. That is a reason to invest in documentation that has nothing to do with humans reading it.

Q3 — A refinement, and what it says about shelf life

Read the two claims precisely and they do not contradict. June 2025: do not let parallel subagents make independent decisions, because actions carry implicit decisions and conflicting decisions produce incoherent results. April 2026: multiple agents may contribute **intelligence** — review, escalation, decomposition — provided **writes stay single-threaded**. The 2026 rule is the 2025 diagnosis with a valve fitted: separate the thing that was dangerous (uncoordinated mutation) from the thing that was merely assumed to be dangerous (multiple models being involved). Some of the change is capability drift — Yan says models became more naturally agentic — but the load-bearing part is a sharper distinction, and distinctions age far better than benchmarks do. So the shelf-life lesson is not “ignore 2025 advice”. It is: separate a piece of advice into its **mechanism** and its **calibration**.

- **Mechanisms — long half-life.** Conflicting decisions produce incoherent results · observations are the only external correction · cost grows with the square of turns.
- **Calibrations — stale within months.** How many steps a model can sustain before it drifts · what a token costs.

When you read anything in this field — including the five pre-read briefs for this class — ask which half you are reading. **We come back to this in Capsule 3**, when we ask which parts of this week survive contact with next year.

Q4 — Why a clean context sees more

The original agent has, sitting in its context, every justification it wrote for every choice it made. Those tokens condition everything it produces next, so asking it to review its own work is asking it to argue against a document it has just spent thirty turns writing — and it is a system built to continue text coherently. A fresh agent given only the diff and the requirements has no such commitment, and its attention budget (Reading 1) is spent entirely on the artefact rather than on the history of its production. Cognition’s reported numbers — roughly two bugs per pull request, 58% of them severe — are strong enough to be worth copying even discounted. The pattern generalises anywhere the producer is context-poisoned by its own process: a second agent checking whether a report’s claims are supported by the retrieved sources; a fresh evaluator scoring a customer reply against policy; a clean-slate pass asking “does this actually answer the original question?” after a long agent run. Two design notes if you use it in A2. Give the reviewer the **artefact and the requirements, not the trace** — the trace re-imports the contamination you were trying to escape. And keep the reviewer read-only: it contributes intelligence, the writes stay single-threaded, which is the 2026 rule doing real work.

Q5 — The five questions

Our reading — these five are ours, assembled from the week’s failure modes rather than from a generic procurement list. Ask them in this order:

1. **What is this evaluated on — and can I see the eval set?**
2. **Show me it working on an existing process or codebase of ours**, not a new one.
3. **What happens when it fails** — how does it fail, how do I find out, and how quickly?
4. **What does a failed run cost me** — in tokens, and in the human time to unpick it?
5. **How does the system tell me it is unsure** — and what does it do at that moment?

The one that most changes the decision is **question 3**, and it is the one vendors are least prepared for. Capability claims can be verified later with a pilot; failure behaviour is architectural, and by the time you discover it you have already deployed. A system that fails loudly, cheaply and reversibly with a 70% success rate is worth more than one that fails silently with 85%, because the first has a workflow around it and the second has an incident. Notice that Devin 2.1’s confidence indicators are precisely an answer to question 5, shipped ten months after the demo — and that the whole arc of this reading is the industry learning to answer question 3 after being asked question 1 too many times.

APPENDIX — EVALUATION VOCABULARY, EXPLAINED

*The distinction that carries this reading: **capability measured on someone else’s curated tasks, versus usefulness measured on yours**. Every term below helps you keep the two apart.*

SWE-bench

- **Plain.** A test set of real GitHub issues; the agent must produce a patch that makes the repository’s tests pass.
Technically. Tasks come with a reproducible environment and a decisive automated test — which supplies the ground-truth signal an agent needs by construction, and is exactly why performance on it does not transfer to

underspecified work. **SWE-bench Verified** is the human-validated subset; frontier scores moved from roughly 40% to over 80% in the year to early 2026.

Assisted vs unassisted

- **Plain.** Were you told which files to change, or did you have to work that out too?

Technically. The 2024 comparison was 13.86% unassisted against 1.96% assisted — a genuinely harder setting for Devin, and a rare case of a vendor benchmark being conservative in that particular respect.

Field evaluation

- **Plain.** Twenty of your own real tasks, judged by the people who would otherwise have done them.

Technically. Answer.AI's method (Jan 2025): 3 successes, 14 failures, 3 inconclusive, disaggregated by task type — the disaggregation is what made it useful, because the aggregate rate would have hidden the greenfield/brownfield split entirely.

Greenfield vs brownfield

- **Plain.** Starting from nothing, versus changing something that already exists and that other things depend on.

Technically. Brownfield constraints are largely unwritten, so they are unavailable to retrieval and invisible to the agent. Empirically the sharpest predictor of agent success in the Answer.AI data: 2/8 greenfield, 0/8 brownfield.

Autonomy slider

- **Plain.** How much the agent may do before a human looks — a dial, not a switch.

Technically. Positions include suggest-only, propose-then-approve, execute-with-checkpoints, and fully autonomous with post-hoc review. The 2024 product assumed the far end; every subsequent release added positions in the middle.

Single-threaded writes

- **Plain.** Many agents may advise; only one may change anything.

Technically. Cognition's 2026 rule. Preserves the coherence that parallel mutation destroys, while keeping the benefit of extra perspectives — the review-agent, smart-friend and manager patterns all obey it.

Review agent

- **Plain.** A second agent, with no memory of how the work was done, checks the work.

Technically. Clean context avoids the producing agent's self-justification and spends its attention budget on the artefact. Reported at roughly 2 bugs per PR, 58% severe. Give it the artefact and the requirements — not the producing agent's trace.

Confidence signalling

- **Plain.** The agent tells you how sure it is, so you know what to check.

Technically. Shipped in Devin 2.1 (May 2025) as per-task indicators. Converts a binary "done" into a triage signal, which is what makes a 70%-reliable agent usable inside a human workflow.

SOURCE LINKS

Cognition — "Introducing Devin, the first AI software engineer" (12 Mar 2024) · cognition.com/blog/introducing-devin

Answer.AI — "Thoughts On A Month With Devin" (8 Jan 2025) · www.answer.ai/posts/2025-01-08-devin.html (the independent evaluation)

Cognition — "Don't Build Multi-Agents" (12 Jun 2025) · cognition.com/blog/dont-build-multi-agents

Cognition — "Multi-Agents: What's Actually Working" (22 Apr 2026) · cognition.com/blog/multi-agents-working (the refinement)

Companion · **Reading 1** is the restraint this story is a case study in; **Reading 4** is the same product bet, made two years later with the lessons applied.

CAVEATS

1. This summary is not a substitute for the three sources. The Answer.AI post in particular is worth reading in full — the task-by-task detail is where the learning is.
2. Answer.AI's evaluation is one team, twenty tasks, in January 2025, on a product that has changed substantially since. It is evidence, not a verdict, and it is not a fair description of Devin in 2026.
3. Cognition's 2026 figures (roughly 2 bugs per PR, 58% severe; ~8× enterprise growth in six months) are self-reported by the vendor and not independently verified. The pattern is worth copying; the numbers are worth discounting.
4. This summary was generated and fact-checked using Anthropic models — a competitor in this market. The "claim, evidence, correction" framing, the autonomy-slider reading and the five vendor questions are the instructor's. Flag anything that looks wrong.